

Licenciatura em Engenharia Informática

Programação Aplicada

Tabelas de Requisitos

Elaborado em: 2026/06/02

Atualizado em: 2026/06/03

Nome e número do(s) Aluno(s):

António Dinis Myroshnyk (nº 2021157297)

Mariana Pais Jorge Magalhães (nº 2022147454)

Proj 1

Requisito	Descrição	Estado
R1	Permitir aos utilizadores registarem-se e autenticarem-se na aplicação	Feito
R2	Permitir o acesso por 3 tipos de utilizadores: gestores, clientes e funcionários	Feito
R3	Utilizadores caracterizados por nome, username, password, estado, email e tipo	Feito
R4	Username e email devem ser únicos	Feito
R5	Email deve apresentar formato válido	Feito
R6	Cada utilizador apenas pode alterar a sua própria informação	Feito
R7	Gestores podem visualizar e alterar dados de todos os utilizadores e criar gestores	Feito
R8	Clientes e funcionários têm NIF, contacto telefónico e morada. Funcionários têm especialização e data de início. Clientes têm sector e escalão	Feito
R9	NIF deve ser único e possuir 9 dígitos	Feito
R10	Contacto telefónico deve possuir 9 dígitos e iniciar por 9, 2 ou 3	Feito
R11	Clientes e funcionários registam-se na plataforma. Gestores são criados por outros gestores	Feito
R12	Pedidos de registo devem ser notificados aos gestores	Feito

R13	Gestores aprovam ou rejeitam pedidos de registo	Feito
R14	Conta reprovada ou não analisada apresenta mensagem informativa	Feito
R15	Gestor pode inativar ou ativar conta de utilizador	Feito
R16	Utilizador pode solicitar remoção da conta, com notificação ao gestor	Feito
R17	Sem utilizadores criados, aplicação solicita credenciais para criar conta gestor	Feito
R18	Após autenticação apresenta "Bem-vindo [nome utilizador]"	Feito
R19	Ao encerrar apresenta "Adeus [nome utilizador]"	Feito
R20	Sistema gere equipamentos e reparações	Feito
R21	Equipamento pode ter várias reparações ao longo do tempo	Feito
R22	Cliente submete equipamentos e pedidos de reparação	Feito
R23	Cada cliente pode ter vários equipamentos, cada equipamento pertence a um cliente	Feito
R24	Cada reparação está associada a um equipamento e realizada por um funcionário	Feito
R25	Clientes submetem pedidos de reparação	Feito
R26	Pedido de reparação aprovado pelo gestor, que atribui funcionário	Feito
R27	Funcionários gerem o processo de reparação	

R28	Equipamento caracterizado por marca, modelo, SKU, data de fabrico, lote, data de submissão e data de reparação	Feito
R29	SKU único, valor aleatório entre 1 e 1 000 000	Feito
R30	Verificar duplicação de código de modelo ao inserir equipamento	Feito
R31	Número de reparação composto por número sequencial seguido da data no formato AAAAMMDDHHMMSS	Feito
R32	Reparação inclui data de criação, data de fim, tempo decorrido, custo, estado, observações e listagem de testes	Feito
R33	Testes de operacionalidade caracterizados por designação, descrição e data	Feito
R34	Peças usadas na reparação devem ser registadas e quantidade decrementada	Feito
R35	Peça caracterizada por código interno, designação, fabricante e quantidade	Feito
R36	Gestor responsável pela inserção e atualização de peças	Feito
R37	Processo inicia quando cliente solicita reparação	Feito
R38	Gestor notificado, aprova ou rejeita pedido e atribui funcionário	Feito
R39	Funcionário notificado quando processo atribuído, pode aceitar ou rejeitar	Feito
R40	Se funcionário recusar, gestor notificado e designa outro funcionário	Feito

R41	Funcionário inicia e encerra reparação, não tem de ser numa única sessão	Feito
R42	Após finalizado pelo funcionário, gestor arquiva o processo	Feito
R43	Cliente e gestor podem consultar estado da reparação	Feito
R44	Gestor notificado quando novo utilizador se regista	Feito
R45	Gestor notificado quando existe pedido de reparação	Feito
R46	Cliente notificado se pedido de reparação rejeitado	Feito
R47	Funcionário notificado quando pedido atribuído	Feito
R48	Gestor notificado se funcionário rejeitar pedido	Feito
R49	Cliente notificado quando processo de reparação finalizado	Feito
R50	Gestor notificado quando processo passa 10 dias sem ser finalizado	
R51	Não é necessário listar peças menores que 10 unidades	N/A
R52	Listagens por páginas de 10 registos	Feito
R53	Listagens permitem ordenar ascendente ou descendente	Feito
R54	Listagens permitem pesquisas para filtrar resultados	Feito
R55	Pesquisa avançada, apresenta registos com termo parcial	Feito
R56	Gestores listam todos os utilizadores ordenados por nome	Feito

R57	Gestores pesquisam utilizadores por nome, username ou tipo	Feito
R58	Gestores listam pedidos de reparação ordenados por data ou número	Feito
R59	Gestores listam pedidos de reparação não finalizados	Feito
R60	Gestores pesquisam pedidos por número, estado ou cliente	Feito
R61	Gestores pesquisam pedidos por intervalo temporal	
R62	Gestores pesquisam equipamentos por marca ou código	Feito
R63	Clientes listam os seus pedidos ordenados por data ou número	
R64	Clientes pesquisam os seus pedidos por número ou estado	
R65	Clientes listam os seus equipamentos ordenados por marca ou código	
R66	Clientes pesquisam os seus equipamentos por marca ou código	
R67	Funcionários listam os seus pedidos ordenados por data ou número	
R68	Funcionários pesquisam os seus pedidos por número ou estado	
R69	Acesso a base de dados relacional para gerir toda a informação	Feito

R70	Acesso restringido com credenciais armazenadas na base de dados	Feito
R71	Parâmetros de acesso à base de dados armazenados em ficheiro Properties	Feito
R72	Interface em modo texto para interação com o utilizador	Feito
R73	Calcular e apresentar tempo de execução da aplicação	Feito
R74	Manter informação genérica do sistema como data da última execução e número de execuções	Feito
R75	Registo de ações (log) dos utilizadores armazenado na base de dados	Feito
R76	Listar conteúdo do log através da aplicação	
R77	Pesquisar registos por utilizador no log	
R78	Aplicação estruturada segundo paradigma Orientado a Objetos em Java	Feito
R79	Implementar estruturas de armazenamento otimizadas	Feito
R80	Validar todas as leituras de dados do utilizador	Feito
R81	Apresentar mensagens informativas ao utilizador	Feito

Proj 2

Requisito	Descrição	Estado
G1	Completar e corrigir a aplicação iniciada no projeto 1	
G2	Utilizadores possuem foto, que pode ser inserida ou alterada em qualquer momento. Caso não seja definida, deve existir uma imagem por defeito no sistema	
G3	No momento de registo deve ser enviado um email a informar que o registo foi realizado	
G4	Em todos os formulários de inserção de dados deve existir um campo adicional (JTextField ou JAreaField) para inserir observações ou comentários	
G5	Após autenticação, deve surgir uma caixa de diálogo ou zona em realce na janela principal a indicar a existência de notificações, caso existam	
G6	Deve ser possível imprimir um extrato das ações de um processo (datas e utilizadores que realizaram ações)	
G7	Todas as listagens devem incluir barras de deslocamento	
G8	Toda a interação com o utilizador deve ser realizada através de interfaces gráficas (JavaSwing / JavaFX). As interfaces devem ser intuitivas, limpas e claras	
G9	Devem ser usadas componentes gráficas adequadas à informação: JList para listagens simples, JTable para listagens estruturadas	
G10	Os formulários devem possuir tooltips de ajuda sobre cada elemento da interface	

G11	Não é necessário apresentar graficamente mensagens de boas-vindas, de encerramento, nem estatísticas de utilização	N/A
G12	Não é necessário implementar graficamente a manipulação de peças e testes	N/A
G13	Não é necessário gestores poderem criar outros gestores através da interface gráfica	N/A
G14	O código deve estar harmonizado: a mesma funcionalidade deve estar implementada de forma idêntica e consistente ao longo de todo o código	

Singin.java

o que muda?

A lógica do `handleInput()` fica toda, a verificação de credenciais, o `isActiveUser()`, o `instanceof` para saber o tipo. Isso vai para um método chamado por um botão.

O que muda é a apresentação. Em vez de `Input.getInput()` temos campos de texto, e em vez de `System.out.println()` temos mensagens de erro na janela.

Construtor

Configuração da janela principal

Começamos por definir o título da janela.

```
setTitle("Login");
```

Depois definimos o comportamento da aplicação quando o utilizador clica no botão X da janela.

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Valores mais comuns

Valor	Comportamento
<code>JFrame.EXIT_ON_CLOSE</code>	Fecha a janela e termina a aplicação
<code>JFrame.DISPOSE_ON_CLOSE</code>	Fecha apenas esta janela
<code>JFrame.HIDE_ON_CLOSE</code>	Esconde a janela
<code>JFrame.DO_NOTHING_ON_CLOSE</code>	Não faz nada

Para o caso é o `JFrame.EXIT_ON_CLOSE` para após clicar no "x" da janela encerra a aplicação

A seguir definimos o tamanho inicial da janela

```
setSize(350, 250);
```

Sintaxe: `setSize(largura, altura)` Neste caso:

- largura = 350 pixels
- altura = 250 pixels

Resultado:

```
350 px
```

<----->

```
+-----+
|           | 250 px
|           |
|           |
+-----+
```

O `setLocationRelativeTo(null);` posiciona a janela no ecrã, como recebe null a janela aparece centrada.centralizada

Criação do painel principal

Temos

```
JPanel panel = new JPanel(new GridBagLayout());
```

Ele cria um painel para colocar os componentes. A janela (JFrame) é o recipiente principal. Dentro dela colocamos um painel (JPanel).

Estrutura:

```
JFrame
├──
│   ├── JPanel
│   │   ├── Username
│   │   ├── Password
│   │   └── Botões
```

A seguir temos `new GridBagLayout()` que é um gestor de layout baseado numa grelha, que permite posicionar componentes em linhas e colunas.

Exemplo

```
+-----+
| Username | [_____] |
| Password | [_____] |
|           | [Botões] |
+-----+
```

Borda com título

```
panel.setBorder(
    BorderFactory.createTitledBorder(
        BorderFactory.createEtchedBorder(),
        "Login",
        TitledBorder.LEFT,
        TitledBorder.TOP
    )
)
```

Adiciona uma borda ao painel, com o título em cima do lado esquerdo.

O **BorderFactory** é uma classe utilitária para criar bordas.

O `createEtchedBorder()` cria uma borda com efeito 3D.

O `createTitledBorder()` cria uma borda com um título.

```
GridBagConstraints gbc = new GridBagConstraints();
```

Utilizado para indicar onde cada componente será colocado é como um conjunto de instruções para o layout.

Os Insets definem as margens

Sintaxe

Neste caso

10 px em todos os lados

```
gbc.gridx = 0;
gbc.gridy = 0;
```

Posição do componente. O **gridx** é a Coluna, a **gridy** é a linha.

Neste caso:

11

```
+-----+-----+
```

O `anchor`:

```
gbc.anchor = GridBagConstraints.WEST;
```

Alinha o componente à esquerda.

Valor	Posição
WEST	esquerda
EAST	direita
CENTER	centro
NORTH	cima
SOUTH	baixo

Campo de texto Username

```
txtUsername = new JTextField(15);
```

Cria um campo de texto. O valor 15 representa aproximadamente o número de caracteres visíveis.

O `fill`:

```
gbc.fill = GridBagConstraints.HORIZONTAL;
```

Permite que o campo se estique horizontalmente.

Sem isto:

```
[_____]
```

Com isto:

```
[_____]
```

Campo Password

A lógica é a mesma do username mas com uma diferença o `JPasswordField`

```
txtPassword = new JPasswordField(15);
```

Semelhante ao `JTextField`, mas esconde os caracteres.

Resultado:

```
[*****]
```

Painel dos botões

```
JPanel buttons = new JPanel(  
    new FlowLayout(FlowLayout.RIGHT)  
);
```

Cria um painel para organizar os botões.

O `FlowLayout` organiza componentes em sequência. Depois usa o `FlowLayout.RIGHT` para alinhar à direita.

funcionalidades botões

```
JButton btnLogin = new JButton("Login");
```

Cria um botão.

E o `ActionListener` define a ação do botão.

```
btnLogin.addActionListener(  
    e -> handleInput()  
);
```

Quando o botão é clicado:

```
Login  
↓  
handleInput()  
↓  
Validação dos dados  
↓  
Acesso ao sistema
```

Adicionar componentes ao painel

```
panel.add(new JLabel("Username"), gbc);
```

Adiciona o componente ao painel usando as regras definidas em `gbc`. Cada componente é colocado numa posição diferente alterando:

```
gbc.gridx  
gbc.gridy
```

Esquema final

```
+-----+  
| Login |  
+-----+
```

```
| Username [_____] |
| Password [_____] |
|
| [Exit][Login] |
+-----+
```

Adicionar o painel à janela

```
add(panel, BorderLayout.CENTER);
```

Coloca o painel principal dentro da janela.

Tornar a janela visível

```
setVisible(true);
```

Mostra a janela ao utilizador. Sem esta instrução: Janela criada → Mas invisível

Com esta instrução: Janela criada → Janela visível